TCD3
Test Automation & Continuous Delivery

Static Code Analysis

S. Wagner

Checking Code Quality by Inspection

- Code Reviews
  - fundamental principle in agile development
  - very effective for improving code quality
  - improve communication and skills in a team
  - suffer from "human factors" (subjectivity, emotions, etc.)
  - bind many resources and are very expensive

- Static Code Analysis
  - automatically and repeatedly check code by a set of rules (continuous inspection)
  - benefit from findings and best practices of many others
  - rules can be customized to specific needs and processes
  - results are objective and do not have any "human factors"
  - can be integrated into the CI/CD pipeline
  - does not require human resources and is very cheap

Testing vs. Inspection

- Testing
  - software is executed to check its functionality
  - used to find failures, i.e., inabilities of the software to fulfill its requirements
  - tests require more effort, as they have to be written/defined

- Inspection
  - code is analyzed by a set of rules
  - software metrics are calculated (e.g., complexity metrics)
  - helps to identify problems before they turn into failures
  - predefined rules can be used right away
  - rules can cover many different aspects of code quality (e.g., formatting, anti-patterns, naming, complexity, duplication)

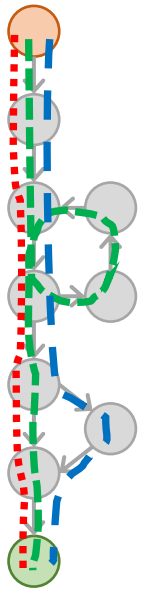
Software Metrics

- Software metrics describe measurable properties of software
  - extend measurement of software quality
  - indicate units which might be problematic
  - visualize quality changes over time

- Metrics can be categorized into several topics:
  - size metrics (SLOC, number of classes, number of methods, etc.)
  - complexity metrics (cyclomatic complexity, cognitive complexity, etc.)
  - test metrics (number of tests, test coverage, etc.)
  - duplications (duplicated lines of code, etc.)

Code Complexity

- Correlation of code complexity and defects
  - long methods (i.e., many lines of code and many different paths) are much more likely to contain defects
  - code complexity and defect rate are directly proportional
- Cyclomatic Complexity
  - number of distinct paths through a method
  - cyclomatic complexity > 10 is considered harmful
  - cyclomatic complexity should correlate with the number of test cases
- How to deal with high cyclomatic complexity?
  - write tests
  - refactor by extracting methods
  - but before you start refactoring, write tests



1 + 1 + 1 = 3

Code Coupling

- Objects with a lot of dependencies are brittle
  - defects can arise in the object, if one of its dependencies change
  - a change in the object can lead to defects in its dependencies (side effects)

- Afferent Coupling (Fan In)
  - number of relationships to the object
  - high afferent coupling means that many other objects depend on that object
  - objects with high afferent coupling should have many unit tests

- Efferent Coupling (Fan Out)
  - number of relationships from the object
  - high efferent coupling means that this objects uses many other objects

- Instability
  - Instability = Efferent Coupling / (Efferent Coupling + Afferent Coupling)
  - indicates an object's resilience to change (0 = stable, 1 = instable)

Insta-bility		Efferent Coupling (Fan Out)							
		10	20	30	40	50	60	70	80
Afferent Coupling (Fan In)	10	0,50	0,67	0,75	0,80	0,83	0,86	0,88	0,89
	20	0,33	0,50	0,60	0,67	0,71	0,75	0,78	0,80
	30	0,25	0,40	0,50	0,57	0,63	0,67	0,70	0,73
	40	0,20	0,33	0,43	0,50	0,56	0,60	0,64	0,67
	50	0,17	0,29	0,38	0,44	0,50	0,55	0,58	0,62
	60	0,14	0,25	0,33	0,40	0,45	0,50	0,54	0,57
	70	0,13	0,22	0,30	0,36	0,42	0,46	0,50	0,53
	80	0,11	0,20	0,27	0,33	0,38	0,43	0,47	0,50

Code Duplication

- Copy & Paste Programming
  - code blocks are simply copied without thinking about better ways to reuse or to generalize
  - is often not even noticed
- Problems of duplicated code
  - higher maintenance costs
  - increased testing effort
- Static code analysis can easily identify duplicated code

Code Coverage

- Code coverage indicates how much code is executed by automated tests
  - different levels of granularity
    - class coverage, method coverage, line (or statement) coverage, branch (or path) coverage
- Coverage data is retrieved by sampling or instrumenting code while tests are executed
  - can have a significant impact on the performance of test execution
  - coverage reports are usually not needed as immediate feedback, as they show trends in the long run
  - analyzing coverage might be postponed to nightly builds, if performance is an issue

SonarQube

https://sonarqube.org

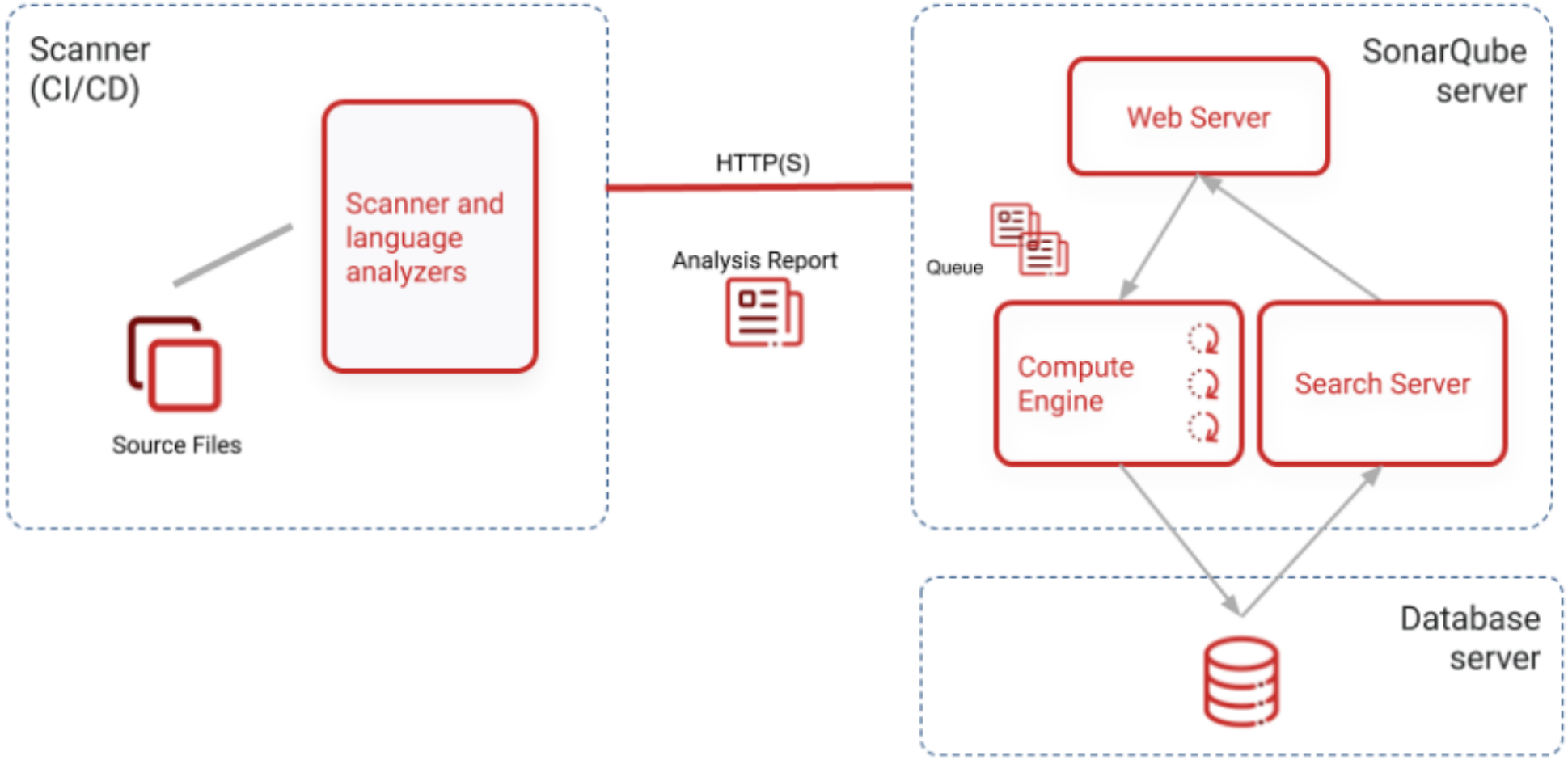


- SonarQube is a web-based open-source tool for continuously inspecting code quality
  - supports 25+ programming languages (e.g., Java, C#, C/C++, JavaScript, TypeScript, Python, Go, PHP, etc.)
  - detects bugs, code smells, security vulnerabilities
  - provides reports on test coverage and code duplication

- SonarQube is maintained by SonarSource
  - development started in 2006
  - company SonarSource was founded in 2008
  - located in Switzerland
  - received $45 million USD venture capital in 2016

SonarQube – Architecture

https://sonarqube.org



https://docs.sonarqube.org/latest/setup/install-server

SonarQube – Test Coverage

https://sonarqube.org

- SonarQube does not execute tests itself
- Separate coverage tool is required to provide coverage data
  - e.g., Coverlet, a cross platform code coverage tool for .NET https://github.com/coverlet-coverage/coverlet
  - computes line, branch, and method coverage
  - coverage data can be created in various formats, e.g., opencover, which is also supported by SonarQube

```
<?xml version="1.0" encoding="utf-8" ?>
<RunSettings>
  <DataCollectionRunSettings>
    <DataCollectors>
      <DataCollector friendlyName="XPlat code coverage">
        <Configuration>
          <Format>opencover</Format>
        </Configuration>
      </DataCollector>
    </DataCollectors>
  </DataCollectionRunSettings>
</RunSettings>
```

```
C:\Game2048>dotnet test --collect:"XPlat Code Coverage" --settings coverlet.runsettings
  Determining projects to restore...
  Restored C:\Game2048\Game2048\Game2048.csproj (in 87 ms).
  Restored C:\Game2048\Game2048Web\Game2048Web.csproj (in 87 ms).
  Restored C:\Game2048\Game2048Tests\Game2048Tests.csproj (in 368 ms).
  Game2048 -> C:\Game2048\Game2048\bin\Debug\net6.0\Game2048.dll
  Game2048Tests -> C:\Game2048\Game2048Tests\bin\Debug\net6.0\Game2048Tests.dll
Test run for C:\Game2048\Game2048Tests\bin\Debug\net6.0\Game2048Tests.dll (.NETCoreApp,Version=v6.0)
Microsoft (R) Test Execution Command Line Tool Version 17.1.0
Copyright (c) Microsoft Corporation.  All rights reserved.

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Attachments:
  C:\Game2048\Game2048Tests\TestResults\b4473a7b-eb43-4045-81b8-133d5b9ba473\coverage.opencover.xml
Passed!  - Failed:     0, Passed:    31, Skipped:     0, Total:    31, Duration: 59 ms - Game2048Tests.dll (net6.0)

C:\Game2048>
```

SonarQube – Pipeline Integration



https://sonarqube.org

- Integration of SonarQube into the build process is very easy
  - program SonarScanner is executed during the build, which reports analysis results to the SonarQube server
  - several build automation tools are directly supported (e.g., MSBuild, Maven)
  - SonarScanner is also provided as a Docker container (nosinovacao/dotnet-sonar)

- SonarQube in a .NET build:

```
dotnet /sonar-scanner/SonarScanner.MSBuild.dll begin /d:sonar.login="$sonar_login"
                                                     /k:"$sonar_project"
                                                     /d:sonar.host.url="$sonar_url"
                                                     /d:sonar.cs.opencover.reportsPaths="**/coverage.*.xml"

dotnet build

dotnet /sonar-scanner/SonarScanner.MSBuild.dll end   /d:sonar.login="$sonar_login"
```

  - `/d:sonar.host.url` = URL of the SonarQube server
  - `/k:` = project ID
  - `/d:sonar.login` = access token generated for the project
  - `/d:sonar.cs.opencover.reportsPath` = path to retrieve coverage reports from

SonarQube – Rules & Quality Profiles



https://sonarqube.org

- Rules
  - static code analysis of SonarQube is based on rules
    - e.g., 639 rules for Java in SonarQube 8.9.1
    - 4 categories: bugs, vulnerabilities, code smells, security hotspots
  - rules can be extended by plugins from the Sonar Marketplace
    - rules from other analyzers can be integrated (e.g., Checkstyle, Findbugs)
  - custom rules can also be integrated as plugins

- Quality Profiles
  - rules can be activated or deactivated as needed
  - quality profile represents the set of active rules and is defined per language
  - quality profiles can be set individually for each project
  - arbitrary many custom quality profiles can be defined

SonarQube – Quality Gates

https://sonarqube.org

- Quality gates represent conditions which have to be fulfilled by a project to be acceptable
  - ensure minimum standards for specific measures
  - conditions can be defined only for new code or for all code
- Arbitrary many custom quality gates can be defined
- Each project is associated with one quality gate
- Default quality gate in SonarQube 8.9.1:

Conditions on New Code

Metric	Operator	Value
Coverage	is less than	80.0%
Duplicated Lines (%)	is greater than	3.0%
Maintainability Rating	is worse than	A
Reliability Rating	is worse than	A
Security Hotspots Reviewed	is less than	100%
Security Rating	is worse than	A

SonarQube – Screenshot

https://sonarqube.org

